CENTRO UNIVERSITARIO UNIFECAF

SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS EAD

WEB PROGRAMMING FOR FRONT END

Wellington Diego Santos da Silva

160392

Tutor: **Fernando Leonid**

Cotia – S.P

2026

Wellington Diego Santos da Silva

R.A: 160392

Criação de uma aplicação front-end para uma API de desenhos de sucesso dos anos 90 e 2000.

Trabalho apresentado como requisito parcial de avaliação da disciplina **WEB PROGRAMMING FOR FRONT END** do Curso de Graduação em **SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS EAD** do Centro Universitário UniFECAF.

Tutor: **Fernando Leonid**

Cotia – S.P

2026

1. O que é uma API e como funciona o consumo de dados em tempo real

Uma API (Application Programming Interface) é um conjunto de regras e protocolos que permite que diferentes sistemas se comuniquem. No contexto web, uma API pública expõe dados através de endpoints HTTP que retornam respostas em formatos como JSON. O consumo em tempo real, neste projeto, ocorre quando o JavaScript faz uma requisição fetch ao servidor da API e recebe os dados atualizados instantaneamente, sem necessidade de recarregar a página inteira. Esse modelo é chamado de requisição assíncrona: a página continua interativa enquanto os dados são buscados em segundo plano.

2. Conceito de DOM Manipulation e criação de elementos dinâmicos

O DOM (Document Object Model) é a representação em árvore de todos os elementos de uma página HTML. A manipulação do DOM via JavaScript permite criar, modificar e remover elementos após a página já ter sido carregada. Neste projeto, os cards dos personagens não existem previamente no HTML: eles são criados um a um com document.createElement(), têm seus atributos e textos definidos, e são inseridos no container através de appendChild(). Essa técnica é fundamental para aplicações modernas que exibem dados dinâmicos vindos de APIs.

3. Funções básicas utilizadas para consumo da API

- fetch(): inicia uma requisição HTTP assíncrona ao endpoint da API.
- .then(): encadeia o processamento da resposta. O primeiro .then() converte a resposta para JSON com response.json(); o segundo recebe os dados prontos.
- document.createElement(tag): cria um novo elemento HTML na memória.
- appendChild(elemento): insere um elemento como filho de outro, construindo a árvore DOM.
- textContent: define o texto de um elemento de forma segura, sem interpretar HTML.
- classList.add(): adiciona classes CSS para estilização.
- .catch(): captura erros em qualquer etapa da cadeia de Promises.

4. Motivo da escolha da API Rick and Morty

A Rick and Morty API foi escolhida por atender integralmente aos requisitos do trabalho: é pública, gratuita, não exige token de acesso nem cadastro, e fornece dados ricos sobre personagens de um desenho de grande sucesso. A estrutura de resposta é simples e previsível, o que facilita o desenvolvimento.

5. Regras de acesso e documentação da API

- **Autenticação**: não requer token nem chave de API.
- **Limite de requisições**: 10.000 requisições por IP por dia. Se excedido, a API retorna HTTP 429 por 12 horas .
- **Paginação**: a API retorna até 20 personagens por página por padrão.
- **Estrutura de resposta**: o JSON contém um objeto com info (contagem, páginas, próxima/anterior) e results (array de personagens). Cada personagem possui: id, name, status, species, type, gender, origin (objeto com name e url), location, image, episode (array de URLs), url e created.

6. Links

📁 **Repositório:**

https://github.com/Wellingtondsilva/trabalho-uniFecaf-WEB-PROGRAMMING-FRONT-END.git

🚀 **Site Online (GitHub Pages):**

https://wellingtondsilva.github.io/trabalho-uniFecaf-WEB-PROGRAMMING-FRONT-END/

▶️ **YouTube:**

https://youtu.be/TpXS5HiTJeM